Universidad Nacional Autónoma de México

Facultad de Ingeniería

MPOO-2027-1 Modelos de Programación Orientada a Objetos

Práctica 7.

Integrantes del equipo:
León Acosta Diego
Leon Carrasco Jonatan Abdel
Castillo Vizcarra Ihsahn
Fecha de entrega: 16 de octubre de 2026

1. **MainAnimales.java**

```
$ java MainAnimales
Lomito, edad 0 dice ...
Firulais, edad 3 criollo dice Guau
Max, edad 5 labrador asiste a Marta dice Guau
Michi, edad 2 de interior dice Miau
Max es labrador, edad 5
```

2. **MainPersonajes.java**

```
Aldeano | vida 100 ataca con 10
Brakus | vida 80 | fuerza 15 ataca con 25
Lunara | vida 0 | fuerza 12 | fe 40 ataca con 42
Zafiro | vida 60 | mana 50 ataca con 25
Lunara tiene fuerza 12 y fe 40
```

3. **Orden de construcción**

```
$ java MainPersonajes
Personaje listo
Personaje listo
Caballero listo
Personaje listo
Caballero listo
Paladin listo
Personaje listo
Aldeano | vida 100 ataca con 10
Brakus | vida 80 | fuerza 15 ataca con 25
Lunara | vida 0 | fuerza 12 | fe 40 ataca con 42
Zafiro | vida 60 | mana 50 ataca con 25
Lunara tiene fuerza 12 y fe 40
```

AL instanciar lunara, el orden de los mensajes en la consola fue:

1. Personaje listo
2. Caballero listo
3. Paladin listo

¿Por qué?
En Java la herencia funciona como una cadena de construcción de arriba a abajo, cuando se crea un objeto de una clase hija, su constructor invoca primero al constructor de su clase padre usando super().
Para crear a Lunara, se manda a llamar al constructor de Paladín, pero como Paladín hereda de caballero, se le cede el turno a Caballero. A su vez, Caballero hereda de Personaje cediéndole el turno a Personaje.

¿Por qué al crear Brakus salen 2 líneas y al crear Lunara 3?

Porque Brakus es un objeto de tipo Caballero el cual solo tiene 2 niveles en su jerarquía, por lo que la cadena de llamadas a super() es un eslabón más largo e involucra 3 constructores, en cambio Lunara es de tipo paladin, situandose un nivel más abajo en la jerarquía por lo que la cadena de llamadas es un poco mas larga e involucra 3 constructores.

Dificultades y soluciones

1. Omision de la anotacion @Override y formato en el metodo toString()

¿Qué nos pasó?Al imprimir en consola el objeto Lunara, la salida no mostraba el atributo específico de su clase (su nivel de fe), sino únicamente los datos generales heredados de Caballero.

Solución: Al investigar nos percatamos de que no habíamos sobreescrito correctamente el método toString() en la clase Paladín. Le agregamos la notación @Override para asegurarnos de sobreescribir el método base y utilizamos super.toString() para reciclar el formato de la clase padre y concatenar la fe.

2. Error de sintaxis por no colocar superen la primera línea del constructor

¿Qué nos pasó?:Intentamos realizar validaciones e inicializar variables propias de la clase hija antes de llamar al constructor de padre dentro de Paladín, pero el compilador en VS marcaba error de sintaxis en la línea de super().

Solución: Investigando revisamos la regla fundamental de constructores en Java: la llamada a super() debe ser de manera escrita la primerísima línea de código dentro del constructor derivado. Movimos la llamada al inicio y el código compilo sin errores.